

PRÁCTICA N.º 3 — ENUNCIADO

Construcción y consumo de servicios REST

Diseño de recursos, códigos de estado, validación, paginación, versionado y contrato OpenAPI

Asignatura	COM-600 · Microservicios · Gestión 2/2026
Tema del programa	Tema 4 — Comunicación entre Microservicios
Modalidad	Parte 1 guiada en laboratorio · Parte 2 individual
Duración estimada	Parte 1: 4 horas · Parte 2: trabajo autónomo
Puntaje	100 puntos (40 + 60)
Carpeta de trabajo	practica3-rest

Nota: Este documento es el ENUNCIADO: contiene los comandos, las explicaciones y los ejercicios. Las capturas y las respuestas NO se escriben aquí, sino en el documento complementario «Hoja de Evidencias», que es el que se entrega.

1. Objetivos

Al finalizar esta práctica el estudiante será capaz de:

- Diseñar los recursos de una API como sustantivos y expresar la acción con el verbo HTTP, no con la URL.
- Responder con el código de estado que corresponde a cada situación, y no con 200 para todo.
- Validar la entrada antes de tocar la base de datos y rechazar lo inválido con un mensaje útil.
- Devolver todos los errores del servicio con un único formato, estable y documentado.
- Paginar, filtrar y ordenar una colección grande sin obligar al cliente a descargarla entera.
- Versionar la API para poder evolucionarla sin romper a quien ya la consume.
- Publicar el contrato del servicio con OpenAPI y probarlo desde la interfaz generada.
- Empaquetar el servicio y su base de datos con Docker Compose, reutilizando lo aprendido en la Práctica 2.

2. Qué necesita saber antes de empezar

El contenido conceptual de esta práctica se desarrolla en **Tema 4 — Comunicación entre Microservicios**. Aquí solo se recuerda lo mínimo necesario para ejecutar los laboratorios.

- **Recurso:** la cosa que la API expone, nombrada con un sustantivo en plural: /usuarios. Un recurso tiene identidad propia y una URL que lo señala.
- **Verbo HTTP:** la acción sobre el recurso: GET lee, POST crea, PUT reemplaza, PATCH modifica una parte, DELETE elimina. La acción va en el verbo, nunca en la URL.
- **Código de estado:** el número de tres dígitos con el que el servidor resume qué pasó: 2xx salió bien, 4xx el cliente se equivocó, 5xx el servidor falló.
- **Idempotencia:** propiedad de una operación que produce el mismo estado final se ejecute una o cien veces. GET, PUT y DELETE son idempotentes; POST no lo es.
- **Contrato:** la descripción formal de rutas, parámetros, cuerpos y respuestas que el servicio promete cumplir. Aquí se escribe con OpenAPI y se versiona junto al código.

Repaso: La discusión conceptual —estilo REST frente a RPC, el modelo de madurez de Richardson, HATEOAS, comparación con GraphQL y gRPC— corresponde al Tema 4. Esta práctica es la ejecución: aquí se construye y se consume el servicio.

3. Requisitos previos

Requisito	Verificación	Observación
Node.js 20 o superior	node --version	La versión 20 o la 22 LTS
npm 10+	npm --version	Viene con Node
Docker Desktop o Engine 24+	docker --version	Se usa desde el Laboratorio 0 para la base de datos
Cliente HTTP de línea de comandos	curl --version	Postman es válido, pero las capturas piden la salida de curl con -i
MongoDB de la Práctica 1	docker run mongo:7	No se instala en el equipo: se levanta en contenedor, como en la Práctica 2
Git	git --version	Para subir la entrega al repositorio

Atención: Cree una carpeta de trabajo llamada practica3-rest y realice TODO el trabajo dentro de ella. Esa carpeta se sube al repositorio al final.

Los comandos se muestran precedidos por el símbolo \$. NO escriba el símbolo \$ al copiar el comando.

Esta práctica encadena con las dos anteriores: la base de datos es la MongoDB que usted ya manejó en la Práctica 1, y el empaquetado final usa el Dockerfile y el docker compose de la Práctica 2. No empiece de cero lo que ya sabe hacer.

Nunca escriba la contraseña ni la cadena de conexión dentro del código: van en variables de entorno desde el Laboratorio 4.

4. Estructura y forma de entrega

La práctica se apoya en dos documentos:

Documento	Para qué sirve	¿Se entrega?
Enunciado (este archivo)	Guía de trabajo: comandos, explicaciones, ejercicios y rúbrica. Se consulta mientras se trabaja.	No
Hoja de Evidencias	Formulario donde se pegan las 18 capturas y se responden las 8 preguntas de comprobación de la Parte 1.	Sí, en PDF

El trabajo tiene dos partes obligatorias:

Parte	Contenido	Puntaje
PARTE 1 — Guía	7 laboratorios paso a paso. Se ejecutan los comandos y se documenta cada resultado en la Hoja de Evidencias.	40 pts
PARTE 2 — Ejercicios propuestos	6 ejercicios de resolución individual. Se entregan como código en el repositorio más un informe en PDF.	60 pts
TOTAL		100 pts

Entregables

- Hoja de Evidencias en PDF, con las capturas pegadas y las preguntas de comprobación respondidas (Parte 1).
- Repositorio Git público con la carpeta practica3-rest: código del servicio, openapi.yaml, Dockerfile, docker-compose.yml, .env.example, .gitignore y README.md con las instrucciones de ejecución.
- Colección de pruebas (archivo .http, colección de Postman o script de curl) que ejercite cada endpoint y cada código de estado del servicio.
- Informe PDF de la Parte 2 con la resolución de cada ejercicio, las capturas de la evidencia y las justificaciones pedidas.

PARTE 1 — PRÁCTICA GUIADA

Ejecute cada comando en el orden indicado. Cuando aparezca un aviso de «Captura N.º», tome la captura de pantalla y péguela en el recuadro correspondiente de la Hoja de Evidencias. Las «Pregunta N.º» se responden en esa misma hoja, con sus propias palabras: son las que demuestran que entendió lo que ejecutó.

Consejo: Trabaje con los dos documentos abiertos a la vez: este enunciado para leer y ejecutar, y la Hoja de Evidencias para ir registrando. Las capturas y las preguntas están numeradas igual en ambos.

Laboratorio 0 — Entorno y proyecto base

Antes de discutir diseño, hay que tener el servicio respondiendo. En este laboratorio se arma el proyecto, se levanta la base en un contenedor y se comprueba que el servidor contesta.

Paso 1. Verifique las versiones instaladas

Abra una terminal y ejecute los cuatro comandos. Los tres primeros deben responder con un número de versión; el cuarto confirma que tiene un cliente HTTP para probar la API.

```
$ node --version
$ npm --version
$ docker --version
$ curl --version
```

Si curl no está disponible en Windows, use Invoke-WebRequest de PowerShell o Postman, pero tenga presente que las capturas piden ver la línea de estado de la respuesta.

Captura N.º 1: salida de los cuatro comandos anteriores en su terminal. → *péguela en la Hoja de Evidencias.*

Pregunta N.º 1: La Práctica 1 trabajó MongoDB desde el shell y la Práctica 2 empaquetó una API con Docker. ¿Qué agrega esta práctica que ninguna de las dos resolvía todavía? → *respóndala en la Hoja de Evidencias, con sus propias palabras.*

Paso 2. Cree el proyecto e instale las dependencias

Trabajaremos sobre un recurso conocido: usuarios, con los campos nombre, correo y edad. Es el mismo recurso de la guía original de la materia, ahora construido con criterio de diseño.

```
$ mkdir practica3-rest && cd practica3-rest
$ npm init -y
$ npm install express mongodb dotenv
$ npm install --save-dev nodemon
# express: servidor HTTP · mongodb: cliente oficial · dotenv: variables de entorno
```

Abra package.json y agregue el script de arranque en modo desarrollo: "dev": "nodemon src/server.js".

Paso 3. Levante la base de datos en un contenedor

La base no se instala en su equipo. Se levanta como contenedor, exactamente con la mecánica de la Práctica 2. El volumen conserva los datos aunque el contenedor se destruya.

```
$ docker volume create datos-practica3
$ docker run -d --name mongo-practica3 \
  -p 27017:27017 \
  -v datos-practica3:/data/db \
  mongo:7
```

```
$ docker ps
```

Enlace con la Práctica 2: El puerto 27017 se publica solo para que usted pueda inspeccionar la base con mongosh durante el laboratorio. En el Laboratorio 6, cuando la API viva dentro de la misma red de Docker Compose, esa publicación deja de ser necesaria.

Captura N.º 2: salida de docker ps mostrando el contenedor mongo-practica3 en estado Up. → *péguela en la Hoja de Evidencias.*

Paso 4. Escriba el servidor mínimo y compruébelo

Cree la carpeta src/ con dos archivos separados: app.js define la aplicación y server.js es el único que abre el puerto. Esta separación es la que permitirá probar la API sin levantar el servidor en la Práctica 7.

```
src/app.js
const express = require("express");

const app = express();
app.use(express.json());

app.get("/salud", (_req, res) => res.json({ estado: "arriba" }));

module.exports = app;
```

Cree src/server.js con tres líneas: importe app, lea el puerto de process.env.PORT con 3000 por omisión y llame a app.listen. Arranque con npm run dev y pruebe con curl -i http://localhost:3000/salud.

Captura N.º 3: salida de curl -i http://localhost:3000/salud, donde se lea la línea HTTP/1.1 200 OK y el cuerpo JSON. → *péguela en la Hoja de Evidencias.*

A partir de aquí el servicio ya responde. Todo lo que sigue es diseño: qué rutas expone, qué contesta y cómo se documenta.

Laboratorio 1 — Diseño del recurso: rutas, verbos y lo que no es REST

La mitad de los errores de una API se cometen antes de escribir código, al elegir las URL. Aquí se decide el diseño y después se implementa la primera ruta.

Paso 1. Escriba la tabla de rutas antes que el código

Un recurso se nombra con un sustantivo en plural. La colección tiene una URL; cada elemento tiene la suya. La acción la aporta el verbo HTTP. Copie esta tabla al README del proyecto: es la primera versión del contrato.

```
diseño del recurso usuarios
GET    /usuarios      lista la colección      200
POST   /usuarios      crea un usuario         201 + cabecera Location
GET    /usuarios/:id  obtiene uno             200 o 404
PUT    /usuarios/:id  reemplaza uno completo  200 o 404
PATCH /usuarios/:id  modifica parte de uno   200 o 404
DELETE /usuarios/:id  elimina uno             204 o 404
```

Regla práctica: Si al leer la URL en voz alta aparece un verbo, el diseño está mal. La URL nombra cosas; el método HTTP dice qué se hace con ellas.

Paso 2. Reconozca lo que no es REST

Las rutas de la izquierda son las que aparecen en la mayoría de los proyectos que llegan a esta materia. Son HTTP, pero no son REST: convierten la URL en el nombre de una función remota y obligan al cliente a aprenderse un vocabulario nuevo por cada servicio.

antes y después		# Sí lo es
# NO es REST		
GET /obtenerUsuarios		GET /usuarios
GET /getUsuarioById?id=7		GET /usuarios/7
POST /crearUsuario		POST /usuarios
POST /actualizarUsuario		PUT /usuarios/7
POST /borrarUsuario?id=7		DELETE /usuarios/7
GET /usuarios/eliminar/7		DELETE /usuarios/7

La última fila es la más peligrosa: un GET que borra. Cualquier rastreador, precarga del navegador o reintento automático de la red puede ejecutarla sin que nadie la haya pedido.

Paso 3. Implemente la primera ruta con datos en memoria

Todavía no conectamos la base. Primero se estabiliza el diseño con un arreglo en memoria; la persistencia llega en el Laboratorio 4 y no debería obligar a cambiar ni una sola URL.

```
src/usuarios.rutas.js
const { Router } = require("express");

const router = Router();
const usuarios = [
  { id: 1, nombre: "Maria Flores", correo: "maria@usfx.bo", edad: 24 },
  { id: 2, nombre: "Juan Perez", correo: "juan@usfx.bo", edad: 31 },
];

router.get("/", (_req, res) => res.json(usuarios));

module.exports = { router, usuarios };
```

En src/app.js agregue las dos líneas que registran el router: `const { router } = require("./usuarios.rutas");` y `app.use("/usuarios", router);`

Captura N.º 4: salida de `curl -i http://localhost:3000/usuarios` mostrando el 200 y los dos usuarios en JSON. → *péguela en la Hoja de Evidencias.*

Paso 4. Mire la respuesta completa, no solo el cuerpo

La opción `-i` de `curl` imprime la línea de estado y las cabeceras. Una API se juzga por la respuesta entera: el código, el Content-Type y el cuerpo. Compare las dos salidas.

```
$ curl -i http://localhost:3000/usuarios
$ curl -i http://localhost:3000/usuario
# la segunda ruta no existe: observe con qué responde Express
```

Captura N.º 5: las dos salidas de `curl -i`: la ruta correcta con 200 y la ruta inexistente con 404. → *péguela en la Hoja de Evidencias.*

Pregunta N.º 2: La guía anterior de la materia usaba rutas como `/crearUsuario` y `/borrarUsuario`. Elija dos de ellas, reescribálas en estilo REST y explique qué problema concreto evita el cambio para quien consume la API desde otro microservicio. → *respóndala en la Hoja de Evidencias, con sus propias palabras.*

Laboratorio 2 — CRUD completo con los códigos de estado correctos

Un servicio que responde 200 a todo obliga al cliente a leer el cuerpo para saber si algo salió bien. El código de estado existe para evitar eso.

Paso 1. POST: crear y responder 201 con Location

Crear devuelve 201, no 200. Y devuelve la cabecera Location con la URL del recurso recién creado: es la forma en que el servidor le dice al cliente dónde quedó lo que acaba de crear.

```
src/usuarios.rutas.js (agregar)
let siguienteId = 3;

router.post("/", (req, res) => {
  const { nombre, correo, edad } = req.body;
  const usuario = { id: siguienteId++, nombre, correo, edad };
  usuarios.push(usuario);
  res.status(201)
    .location(`/usuarios/${usuario.id}`)
    .json(usuario);
});
```

Pruebe con: `curl -i -X POST http://localhost:3000/usuarios -H "Content-Type: application/json" -d '{"nombre":"Ana Torres","correo":"ana@usfx.bo","edad":22}'`

Captura N.º 6: salida de `curl -i` del POST mostrando HTTP/1.1 201 Created y la cabecera Location con la URL del nuevo usuario. → *péguela en la Hoja de Evidencias.*

Paso 2. GET por identificador: 200 cuando existe, 404 cuando no

Devolver un cuerpo vacío con 200 para un recurso inexistente es un error frecuente: el cliente cree que encontró algo. Si no existe, el código es 404.

```
src/usuarios.rutas.js (agregar)
router.get("/:id", (req, res) => {
  const id = Number(req.params.id);
  const usuario = usuarios.find(u => u.id === id);
  if (!usuario) {
    return res.status(404).json({ mensaje: "Usuario no encontrado" });
  }
  res.json(usuario);
});
```

Pruebe las dos rutas: `curl -i http://localhost:3000/usuarios/1` y `curl -i http://localhost:3000/usuarios/999`

Captura N.º 7: las dos salidas de `curl -i`: el usuario existente con 200 y el inexistente con 404. → *péguela en la Hoja de Evidencias.*

Paso 3. PUT y DELETE: 200 con cuerpo, 204 sin cuerpo

DELETE responde 204 No Content: la operación terminó bien y no hay nada que devolver. Un 204 no lleva cuerpo; si escribe uno, el cliente puede descartarlo.

```
src/usuarios.rutas.js (agregar)
router.put("/:id", (req, res) => {
  const id = Number(req.params.id);
```

```
const i = usuarios.findIndex(u => u.id === id);
if (i === -1) return res.status(404).json({ mensaje: "No encontrado" });
usuarios[i] = { id, ...req.body };
res.json(usuarios[i]);
});

router.delete("/:id", (req, res) => {
  const id = Number(req.params.id);
  const i = usuarios.findIndex(u => u.id === id);
  if (i === -1) return res.status(404).json({ mensaje: "No encontrado" });
  usuarios.splice(i, 1);
  res.status(204).end();
});
```

Captura N.º 8: salida de curl -i del PUT con 200 y del DELETE con 204 No Content sin cuerpo. → *péguela en la Hoja de Evidencias.*

Paso 4. El conflicto: 409 cuando el estado actual impide la operación

Dos usuarios no pueden tener el mismo correo. Eso no es un error de formato (400) ni una ausencia (404): es un conflicto con el estado actual del recurso, y le corresponde el 409.

```
src/usuarios.rutas.js (dentro del POST)
const repetido = usuarios.some(u => u.correo === correo);
if (repetido) {
  return res.status(409).json({ mensaje: "El correo ya está registrado" });
}
```

Ejecute dos veces seguidas el mismo POST y observe la diferencia entre la primera respuesta y la segunda.

Pregunta N.º 3: Ejecute dos veces el mismo POST y después dos veces el mismo DELETE sobre un usuario existente. Compare los códigos obtenidos y explique, con esos resultados, por qué se dice que DELETE es idempotente y POST no lo es. → *respóndala en la Hoja de Evidencias, con sus propias palabras.*

Con esto el CRUD está completo, pero todavía acepta cualquier basura que le manden. Eso se corrige en el siguiente laboratorio.

Laboratorio 3 — Validación de entrada y errores con formato uniforme

Un servicio expuesto en una red recibe peticiones mal formadas todos los días: por descuido, por una versión vieja de un cliente o por alguien probando. Validar es la primera línea de defensa y también la primera cortesía hacia quien consume la API.

Paso 1. Defina un único formato de error para todo el servicio

Si cada endpoint inventa su propio formato, el cliente necesita un caso especial por ruta. Un solo formato, aplicado sin excepción, se programa una vez del lado del consumidor.

```
contrato de error (todas las respuestas 4xx y 5xx)
{
  "error": {
    "codigo": "VALIDACION",
    "mensaje": "La solicitud tiene campos inválidos",
    "detalles": [
      { "campo": "correo", "problema": "formato de correo inválido" },
      { "campo": "edad", "problema": "debe ser un entero mayor que 0" }
    ]
  }
}
```

Criterio: El campo código es para la máquina y no cambia nunca; el campo mensaje es para la persona que depura y puede reescribirse. Nunca incluya la traza de la excepción: eso se registra en el log del servidor, no se envía al cliente.

Paso 2. Escriba el validador del recurso

La validación vive en su propio archivo, separada de la ruta. Devuelve la lista de problemas encontrados; la ruta decide qué hacer con ella.

```
src/usuarios.validacion.js
const CORREO = /^[^\s]+@[^\s]+\.[a-z]{2,}$/i;

function validarUsuario(cuerpo = {}) {
  const d = [];
  const { nombre, correo, edad } = cuerpo;
  if (!nombre || String(nombre).trim().length < 3) {
    d.push({ campo: "nombre", problema: "obligatorio, mínimo 3 caracteres" });
  }
  if (!CORREO.test(String(correo || ""))) {
    d.push({ campo: "correo", problema: "formato de correo inválido" });
  }
  if (!Number.isInteger(edad) || edad <= 0 || edad > 120) {
    d.push({ campo: "edad", problema: "entero entre 1 y 120" });
  }
  return d;
}

module.exports = { validarUsuario };
```

Paso 3. Aplique el validador y centralice la construcción del error

Cree src/errores.js con una función que arme el cuerpo del error, y úsela en todas las respuestas de fallo. Ningún res.status(4xx) debe escribir el JSON a mano.

```
src/errores.js
function fallo(res, estado, codigo, mensaje, detalles) {
  return res.status(estado).json({
    error: { codigo, mensaje, detalles: detalles || [] },
  });
}

// uso dentro del POST:
// const d = validarUsuario(req.body);
// if (d.length) return fallo(res, 400, "VALIDACION",
//   "La solicitud tiene campos inválidos", d);

module.exports = { fallo };
```

Reemplace también los 404 y el 409 del Laboratorio 2 por llamadas a fallo, con los códigos NO_ENCONTRADO y CONFLICTO.

Captura N.º 9: salida de un POST con edad negativa y correo inválido, mostrando el 400 y el cuerpo de error con los dos detalles. → *péguela en la Hoja de Evidencias.*

Paso 4. Compruebe que los tres errores comparten formato

Ejecute las tres peticiones seguidas. La forma del cuerpo debe ser idéntica en las tres; solo cambian el código de estado y el contenido.

```
$ curl -i -X POST http://localhost:3000/usuarios \
  -H "Content-Type: application/json" -d '{"nombre":"A"}'
$ curl -i http://localhost:3000/usuarios/9999
$ curl -i -X POST http://localhost:3000/usuarios \
  -H "Content-Type: application/json" \
  -d '{"nombre":"Ana Torres","correo":"ana@usfx.bo","edad":22}'
# el último, ejecutado dos veces, debe dar 409 la segunda vez
```

Captura N.º 10: las tres respuestas de error (400, 404 y 409) con el mismo formato de cuerpo. → *péguela en la Hoja de Evidencias.*

Pregunta N.º 4: Un compañero propone responder siempre 200 y poner el resultado real dentro del cuerpo, con un campo transaccion true o false. ¿Qué pierde el consumidor de la API con esa decisión? Mencione al menos dos consecuencias concretas. → *respóndala en la Hoja de Evidencias, con sus propias palabras.*

Paso 5. Cierre el manejador central de errores

Si el cliente envía un JSON malformado, express.json lanza una excepción y Express responde con una página HTML de error. Un servicio JSON nunca debe responder HTML. Agregue este middleware al final de app.js, después de todas las rutas.

```
src/app.js (al final)
app.use((_req, res) => {
  res.status(404).json({
    error: { codigo: "RUTA_NO_ENCONTRADA", mensaje: "Ruta inexistente" },
  });
});

app.use((err, _req, res, _next) => {
  const malJson = err.type === "entity.parse.failed";
  console.error(err);
  res.status(malJson ? 400 : 500).json({
    error: {
      codigo: malJson ? "JSON_INVALIDO" : "ERROR_INTERNO",
      mensaje: malJson ? "El cuerpo no es JSON válido" : "Error interno",
    },
  });
});
```

Pruébalo enviando un cuerpo roto a propósito: `curl -i -X POST http://localhost:3000/usuarios -H "Content-Type: application/json" -d '{"nombre":'`

Captura N.º 11: respuesta 400 con el código JSON_INVALIDO al enviar un cuerpo malformado, en formato JSON y no en HTML. → *péguela en la Hoja de Evidencias.*

Laboratorio 4 — Persistencia con MongoDB en contenedor y configuración por entorno

Hasta aquí los datos vivían en un arreglo y desaparecían al reiniciar. Ahora se conecta la base de la Práctica 1, levantada en el contenedor del Laboratorio 0. Ninguna URL debe cambiar: el diseño estaba bien hecho.

Paso 1. Saque la configuración del código

Cree dos archivos: `.env` con los valores reales y `.env.example` con los nombres y valores de ejemplo. El primero se ignora en Git; el segundo se sube, para que otro pueda levantar el proyecto sin adivinar.

```
.env.example
```

```
PORT=3000
MONGO_URL=mongodb://localhost:27017
MONGO_DB=practica3

# .gitignore debe contener, como mínimo:
# node_modules/
# .env
```

Por qué importa: La misma imagen del servicio debe poder correr en su máquina, en la del ayudante y en el servidor sin recompilarse. Lo único que cambia entre esos tres entornos son las variables. Si la cadena de conexión está escrita en el código, no hay imagen portable.

Paso 2. Escriba el repositorio que habla con la base

El repositorio concentra todo el acceso a datos. Las rutas no conocen a MongoDB: solo llaman a estas funciones.

```
src/repositorio.js
const { MongoClient, ObjectId } = require("mongodb");

let col;
async function conectar() {
  const cliente = await new MongoClient(process.env.MONGO_URL).connect();
  col = cliente.db(process.env.MONGO_DB).collection("usuarios");
  await col.createIndex({ correo: 1 }, { unique: true });
  return cliente;
}

const aId = (id) => (ObjectId.isValid(id) ? new ObjectId(id) : null);
module.exports = {
  conectar,
  crear: (u) => col.insertOne(u),
  obtener: (id) => (aId(id) ? col.findOne({ _id: aId(id) }) : null),
  borrar: (id) => col.deleteOne({ _id: aId(id) }),
  porCorreo: (correo) => col.findOne({ correo }),
};
```

Detalle: El índice único sobre correo hace que la base rechace duplicados aunque dos peticiones lleguen al mismo tiempo. La validación en el código evita el caso normal; el índice cubre el caso de carrera.

Paso 3. Conecte al arrancar y compruebe que los datos sobreviven

En src/server.js llame primero a require("dotenv").config() y después a conectar(); solo cuando la conexión esté lista abra el puerto. Reemplace el arreglo en memoria por las funciones del repositorio en cada ruta.

```
$ npm run dev
$ curl -s -X POST http://localhost:3000/usuarios \
  -H "Content-Type: application/json" \
  -d '{"nombre":"Ana Torres","correo":"ana@usfx.bo","edad":22}'
# detenga el servidor con Ctrl+C, vuelva a arrancarlo y consulte de nuevo
$ curl -s http://localhost:3000/usuarios
```

Captura N.º 12: el POST, el reinicio del servidor y el GET posterior mostrando que el usuario sigue existiendo. → *péguela en la Hoja de Evidencias.*

Paso 4. Verifique el dato del lado de la base

Entre al contenedor con mongosh, como en la Práctica 1, y compruebe que el documento está donde debe estar.

```
$ docker exec -it mongo-practica3 mongosh
> use practica3
> db.usuarios.find().pretty()
> db.usuarios.getIndexes()
```

Captura N.º 13: sesión de mongosh mostrando el documento insertado por la API y el índice único sobre correo. → *péguela en la Hoja de Evidencias.*

Pregunta N.º 5: El identificador dejó de ser un número correlativo y pasó a ser el `_id` que genera MongoDB. ¿Qué consecuencia tiene ese cambio para las URL de la API y para el cliente que ya las estaba usando? Relaciónelo con la necesidad de versionar. → *respóndala en la Hoja de Evidencias, con sus propias palabras.*

Laboratorio 5 — Colecciones grandes: paginación, filtrado y ordenamiento

Un GET /usuarios que devuelve todo funciona perfecto con dos usuarios y tumba el servicio con diez mil. La diferencia entre una API de laboratorio y una de producción suele estar exactamente aquí.

Paso 1. Siembre diez mil documentos

Cree scripts/sembrar.js y ejecútelo una sola vez. Necesitamos el problema real antes de resolverlo.

```
scripts/sembrar.js
require("dotenv").config();
const { MongoClient } = require("mongodb");

(async () => {
  const c = await new MongoClient(process.env.MONGO_URL).connect();
  const col = c.db(process.env.MONGO_DB).collection("usuarios");
  const lote = Array.from({ length: 10000 }, (_, i) => ({
    nombre: "Usuario " + i,
    correo: "usuario" + i + "@usfx.bo",
    edad: 18 + (i % 45),
  }));
  await col.insertMany(lote);
  console.log("documentos:", await col.countDocuments());
  await c.close();
})();
```

Ejecute node scripts/sembrar.js y espere el conteo final.

Paso 2. Mida el problema antes de arreglarlo

Pida la colección completa y observe dos cosas: cuánto tarda y cuánto pesa la respuesta.

```
$ curl -s -o salida.json -w "tiempo: %{time_total}s bytes: %{size_download}\n" \
  http://localhost:3000/usuarios
```

Ese peso viaja por la red completo, se carga entero en la memoria del servidor y se vuelve a cargar entero en la del cliente. Con cien clientes simultáneos, el servicio deja de responder.

Captura N.º 14: salida de curl mostrando el tiempo y el tamaño de la respuesta sin paginar. → *péguela en la Hoja de Evidencias.*

Pregunta N.º 6: Con el tiempo y el tamaño que acaba de medir, estime qué ocurriría si veinte clientes pidieran la colección completa a la vez. Explique por qué la paginación es una decisión de diseño del servidor y no una comodidad del cliente. → *respóndala en la Hoja de Evidencias, con sus propias palabras.*

Paso 3. Implemente paginación, filtrado y ordenamiento

Los parámetros de consulta no son parte de la identidad del recurso: filtran y presentan la colección. Fije un tope máximo, porque `limite=1000000` vuelve a ser el problema original.

```
src/usuarios.rutas.js (reemplaza el GET de la colección)
const TOPE = 100;

router.get("/", async (req, res) => {
  const pagina = Math.max(1, Number(req.query.pagina) || 1);
  const limite = Math.min(TOPE, Number(req.query.limite) || 20);
  const filtro = {};
  if (req.query.edadMin) filtro.edad = { $gte: Number(req.query.edadMin) };
  const orden = { [req.query.ordenPor || "nombre"]:
    req.query.orden === "desc" ? -1 : 1 };
  const { datos, total } = await repo.listar(filtro, orden, pagina, limite);
  res.json({
    datos,
    paginacion: { pagina, limite, total,
      paginas: Math.ceil(total / limite) },
  });
});
```

Agregue `listar` al repositorio: aplique `find(filtro).sort(orden).skip((pagina-1)*limite).limit(limite)` y devuelva también `countDocuments(filtro)`.

Paso 4. Compruebe el resultado y el tope

Las tres peticiones prueban la paginación, el filtrado combinado con ordenamiento y la defensa contra un límite abusivo.

```
$ curl -s "http://localhost:3000/usuarios?pagina=2&limite=5" | head -c 400
$ curl -s "http://localhost:3000/usuarios?edadMin=60&ordenPor=edad&orden=desc"
$ curl -s "http://localhost:3000/usuarios?limite=999999" | head -c 200
# el último debe devolver como máximo 100 registros
```

Captura N.º 15: las tres respuestas: la segunda página, el filtro por edad ordenado y el límite recortado al tope de 100. → *péguela en la Hoja de Evidencias.*

La respuesta ya no es un arreglo suelto: es un objeto con `datos` y `paginacion`. Ese cambio rompe a cualquier cliente que esperaba un arreglo, y por eso el laboratorio siguiente empieza por versionar.

Laboratorio 6 — Versionado, contrato OpenAPI y empaquetado con Docker Compose

El servicio funciona. Falta que otro equipo pueda consumirlo sin preguntarle nada a usted, y que arranque en cualquier máquina con un solo comando. Ese es el trabajo de esta sesión final.

Paso 1. Versione la API antes de publicarla

Una API publicada no se cambia: se versiona. El prefijo `/v1` permite que mañana exista `/v2` con otro formato de respuesta mientras los clientes viejos siguen funcionando. Modifique una sola línea de `app.js`.

```
src/app.js (registro de rutas)
// antes:
```

```
// app.use("/usuarios", router);

// después:
app.use("/v1/usuarios", router);
app.get("/salud", (_req, res) => res.json({ estado: "arriba" }));
```

Compruebe que la ruta nueva responde y que la vieja ya no: `curl -i http://localhost:3000/v1/usuarios` y `curl -i http://localhost:3000/usuarios`

Criterio: La ruta de salud queda fuera de la versión a propósito: no es parte del contrato público, es infraestructura, y la usará el healthcheck del contenedor.

Captura N.º 16: las dos respuestas: `/v1/usuarios` con 200 y `/usuarios` con el 404 del manejador central. → *péguela en la Hoja de Evidencias.*

Paso 2. Escriba el contrato en OpenAPI

OpenAPI describe la API en un archivo de texto versionado junto al código. Cree `openapi.yaml` en la raíz. Empiece por la cabecera y la colección.

```
openapi.yaml (primera parte)
openapi: 3.0.3
info:
  title: API de Usuarios
  version: 1.0.0
  description: Servicio REST de la Practica 3 de COM-600
servers:
  - url: http://localhost:3000/v1
paths:
  /usuarios:
    get:
      summary: Lista usuarios con paginacion
      parameters:
        - { name: pagina, in: query, schema: { type: integer } }
        - { name: limite, in: query, schema: { type: integer } }
      responses:
        "200": { description: Pagina de usuarios }
```

Paso 3. Complete el contrato con la creación y el elemento

Continúe el mismo archivo. Cada código de estado que el servicio puede devolver debe estar declarado: si no está en el contrato, para el cliente no existe.

```
openapi.yaml (continuación, mismo nivel de sangría)
  post:
    summary: Crea un usuario
    responses:
      "201": { description: Creado, con cabecera Location }
      "400": { description: Datos invalidos }
      "409": { description: Correo ya registrado }
  /usuarios/{id}:
    get:
      summary: Obtiene un usuario
      parameters:
        - name: id
          in: path
          required: true
          schema: { type: string }
      responses:
        "200": { description: Usuario encontrado }
        "404": { description: No encontrado }
```

Paso 4. Publique la interfaz y pruebe desde ella

La interfaz generada convierte el contrato en documentación navegable y ejecutable. Quien consuma su API no necesitará que usted le explique nada.

```
$ npm install swagger-ui-express ymljs

# en src/app.js, antes del manejador de 404:
# const swaggerUi = require("swagger-ui-express");
# const doc = require("yamljs").load("./openapi.yaml");
# app.use("/docs", swaggerUi.serve, swaggerUi.setup(doc));
```

Abra <http://localhost:3000/docs>, despliegue POST /usuarios, use el botón «Try it out» y envíe un usuario válido y otro con la edad en cero.

Captura N.º 17: interfaz de OpenAPI en el navegador mostrando el resultado de una petición ejecutada desde ella, con su código de estado. → *péguela en la Hoja de Evidencias.*

Pregunta N.º 7: El contrato vive en el repositorio, junto al código. ¿Qué problema aparece cuando el contrato y la implementación se separan, y qué haría usted para que no ocurra en su proyecto integrador? → *respóndala en la Hoja de Evidencias, con sus propias palabras.*

Paso 5. Empaquete el servicio con Dockerfile

Es el mismo Dockerfile de la Práctica 2, ajustado a este servicio. Copie primero los manifiestos y después el código: así el paso de instalación se reutiliza de la caché mientras no cambien las dependencias.

```
Dockerfile
FROM node:20-alpine
WORKDIR /app
COPY package*.json ./
RUN npm ci --omit=dev
COPY . .
ENV PORT=3000
EXPOSE 3000
CMD ["node", "src/server.js"]
```

Cree también un .dockerignore con node_modules, .env y .git, para no meter en la imagen lo que no corresponde.

Paso 6. Levante el servicio y su base con un solo comando

Docker Compose junta la API y la base en una red propia. Dentro de esa red la base se llama por el nombre del servicio, no por localhost: ese es el detalle que más confunde en la primera vez.

```
docker-compose.yml
services:
  api:
    build: .
    ports: ["3000:3000"]
    environment:
      MONGO_URL: mongodb://mongo:27017
      MONGO_DB: practica3
    depends_on: [mongo]
  mongo:
    image: mongo:7
    volumes: ["datos:/data/db"]
```

volumes:
datos:

Ejecute `docker compose up -d --build`, después `docker compose ps` y finalmente `curl -i http://localhost:3000/v1/usuarios` para confirmar que el servicio responde desde el contenedor.

Captura N.º 18: salida de `docker compose ps` con los dos servicios arriba y la respuesta de `curl` al servicio contenedorizado. → *péguela en la Hoja de Evidencias.*

Pregunta N.º 8: Dentro del `compose` la variable `MONGO_URL` apunta a `mongodb://mongo:27017` y no a `localhost`. Explique por qué, y qué ocurriría si dejara `localhost` en esa variable. → *respóndala en la Hoja de Evidencias, con sus propias palabras.*

El servicio queda diseñado, validado, documentado, versionado y empaquetado. En el Tema 4 se verá qué otras formas de comunicación existen entre microservicios y cuándo REST no es la mejor opción.

PARTE 2 — EJERCICIOS PROPUESTOS

Esta parte se resuelve de forma individual. Cada ejercicio se entrega con: (a) el código o los archivos de configuración en el repositorio, (b) las capturas que demuestren su funcionamiento y (c) la explicación pedida, en el informe PDF.

Nota: Todos los ejercicios se resuelven sobre el proyecto integrador de la materia, no sobre el recurso usuarios del laboratorio. Del 1 al 4 son obligatorios para todos; el 5 y el 6 son los que separan una entrega correcta de una sobresaliente.

Ejercicio 1. Diseño de los recursos de su proyecto (10 pts)

Elija dos recursos de su proyecto integrador que tengan relación entre sí (por ejemplo pedidos y sus items, o cursos y sus inscripciones) y diseñe la API completa antes de escribir una sola línea de código.

Se pide:

- Una tabla con verbo, ruta, descripción y todos los códigos de estado posibles de cada endpoint, incluidos los de error.
- El diseño de la relación entre ambos recursos como subrecurso (por ejemplo `/pedidos/{id}/items`) con su justificación.
- Tres URL con verbo tomadas de un proyecto real suyo o de un ejemplo de internet, reescritas en estilo REST, con la explicación de qué mejora cada una.
- La regla que usó para decidir entre PUT y PATCH en su diseño.

Criterio de aceptación: *Ninguna URL del diseño puede contener un verbo. Un compañero debe poder deducir qué hace cada endpoint sin leer una descripción.*

Ejercicio 2. CRUD con los códigos de estado que corresponden (10 pts)

Implemente el CRUD completo del recurso principal que diseñó en el ejercicio 1, respondiendo con el código correcto en cada situación.

Se pide:

- Los seis códigos del laboratorio en uso real: 200, 201 con cabecera Location, 204, 400, 404 y 409.
- Una captura de `curl -i` por cada uno de esos seis códigos, donde se lea la línea de estado.
- La demostración de idempotencia: la misma petición PUT y la misma DELETE ejecutadas dos veces, con los códigos obtenidos en cada intento y el estado final del recurso.

- La explicación de en qué caso concreto de su dominio se produce el 409.

Criterio de aceptación: *El 201 debe traer la cabecera Location apuntando al recurso creado y el 204 no debe traer cuerpo.*

Ejercicio 3. Validación y catálogo de errores del servicio (12 pts)

Dote a su servicio de una validación completa de la entrada y de un catálogo de errores único, documentado y aplicado sin excepción.

Se pide:

- Al menos seis reglas de validación propias del dominio, no solo campos obligatorios: rangos, formatos, fechas no futuras y coherencia entre campos.
- Un catálogo en el README con cada código de error del servicio, su código de estado HTTP y cuándo se produce.
- Un único formato de cuerpo de error aplicado a todos los 4xx y 5xx, incluidos el 404 de ruta inexistente y el 400 de JSON malformado.
- Un script o colección de pruebas que ejercite cada error del catálogo, con la evidencia de su ejecución.
- La justificación de una regla que decidió validar en la base de datos y no en el código, o al revés.

Criterio de aceptación: *Ninguna respuesta de error puede devolver HTML ni la traza de la excepción. Si un error no está en el catálogo, no debe poder producirse.*

Ejercicio 4. Colección grande servida con criterio (10 pts)

Siembre al menos diez mil registros del recurso principal de su proyecto y haga que la API pueda servirlos sin degradarse.

Se pide:

- Paginación con metadatos de página, límite, total y número de páginas, y un tope máximo de tamaño de página aplicado por el servidor.
- Filtrado por al menos dos campos y ordenamiento por al menos dos, combinables entre sí.
- La medición del tiempo y del tamaño de la respuesta antes y después de paginar, con las dos capturas.
- Un índice creado en la base para el campo por el que más se filtra, con la medición que justifique su creación.

Criterio de aceptación: *Una petición con un límite abusivo debe recortarse al tope, no devolver la colección completa ni un error.*

Ejercicio 5. Contrato OpenAPI publicado y versionado (8 pts)

Publique el contrato de su servicio en OpenAPI, expóngalo en una ruta de documentación y demuestre para qué sirve versionar.

Se pide:

- Un openapi.yaml en el repositorio que describa todos los endpoints, sus parámetros, sus cuerpos y todas sus respuestas, incluidas las de error.
- La interfaz de documentación servida por la propia API, con una petición ejecutada desde ella y su captura.
- Todas las rutas del recurso bajo el prefijo /v1, con la ruta de salud fuera de la versión.
- Un cambio incompatible descrito en el informe (por ejemplo renombrar un campo de la respuesta) y la explicación de cómo lo publicaría sin romper a los clientes de /v1.

Criterio de aceptación: *Cada código de estado que su servicio puede devolver debe estar declarado en el contrato. Un código no declarado se considera contrato incompleto.*

Ejercicio 6. Servicio empaquetado y consumido por otro servicio (10 pts)

Deje su servicio levantándose con un solo comando junto a su base de datos, y demuestre que otro servicio puede consumirlo dentro de la misma red, como ocurrirá en el resto de la materia.

Se pide:

- Dockerfile con `.dockerignore` y `docker-compose.yml` que levante el servicio y su base con un único comando, sin pasos manuales previos.
- Toda la configuración por variables de entorno, con un `.env.example` en el repositorio y ninguna credencial en el código.
- Un segundo servicio mínimo, dentro del mismo compose, que consuma un endpoint del primero por el nombre del servicio en la red interna y devuelva una respuesta compuesta.
- Un healthcheck del servicio en el compose y la evidencia de que el segundo servicio espera a que el primero esté sano.
- Un README con las instrucciones exactas para levantar todo en una máquina limpia.

Criterio de aceptación: Se probará clonando su repositorio en una máquina sin nada instalado salvo Docker y ejecutando el comando que documente. Si requiere un paso manual no documentado, el ejercicio no acredita.

Rúbrica de evaluación

Parte 1 — Práctica guiada (40 puntos)

Criterio	Descripción	Pts
Capturas completas	Las 18 capturas están pegadas en la Hoja de Evidencias, son legibles y corresponden a la máquina del estudiante (se ve su usuario o el nombre del equipo).	15
Ejecución correcta	Las salidas muestran el resultado esperado, sin errores sin resolver.	10
Preguntas de comprobación	Las 8 preguntas están respondidas con criterio propio, no copiadas del enunciado.	10
Orden y presentación	Hoja de Evidencias completa, con datos del estudiante y declaración firmada, entregada en PDF y dentro del plazo.	5
Subtotal		40

Parte 2 — Ejercicios propuestos (60 puntos)

Ejercicio	Qué se evalúa	Pts
1. Diseño de los recursos de su proyecto	Tabla completa con códigos de error, subrecurso justificado, tres correcciones explicadas y criterio PUT frente a PATCH.	10
2. CRUD con los códigos de estado que corresponden	Seis códigos correctos con evidencia, Location presente, 204 sin cuerpo, idempotencia demostrada con datos y 409 con caso de dominio real.	10
3. Validación y catálogo de errores del servicio	Seis reglas de dominio, catálogo documentado, formato uniforme sin excepciones, pruebas de cada error y justificación de la decisión de dónde validar.	12
4. Colección grande servida con criterio	Paginación con metadatos y tope, filtros y ordenamientos combinables, mediciones antes y después, índice justificado con datos.	10
5. Contrato OpenAPI publicado y versionado	Contrato completo con respuestas de error, interfaz funcional con evidencia, prefijo de versión aplicado y estrategia de cambio incompatible razonada.	8
6. Servicio empaquetado y consumido por otro servicio	Compose funcional en máquina limpia, configuración por entorno sin credenciales, consumo entre servicios por nombre de red, healthcheck efectivo y README verificable.	10
Subtotal		60

Penalizaciones

Situación	Descuento
URL con verbos (/crearUsuario, /usuarios/eliminar/7) en la entrega de la Parte 2	-5 pts
Responder 200 a operaciones fallidas o usar códigos de estado inventados	-5 pts
Cadena de conexión, usuario o contraseña escritos dentro del código	-5 pts
Endpoint de colección sin paginación ni tope de tamaño	-3 pts
node_modules o el archivo .env subidos al repositorio (falta .gitignore)	-3 pts
Capturas que no corresponden a la máquina del estudiante	Anulación de la Parte 1
Entrega fuera de plazo	Según reglamento de la materia

Anexo — Errores frecuentes y cómo resolverlos

Mensaje o síntoma	Causa probable	Solución
Cannot GET /usuarios (respuesta HTML con 404)	El router no se registró, se registró después del manejador de 404 o el prefijo no coincide (por ejemplo se agregó /v1 y se prueba sin él).	Revise el orden en app.js: primero app.use de las rutas, al final los manejadores de 404 y de error. Confirme el prefijo exacto con curl -i.
req.body llega como undefined o vacío en el POST	Falta app.use(express.json()) o el cliente no envió la cabecera Content-Type: application/json.	Agregue el middleware antes de registrar las rutas y envíe siempre -H "Content-Type: application/json" en curl.
MongoServerError: E11000 duplicate key error	El índice único sobre correo rechazó un duplicado que la validación del código dejó pasar.	Capture ese error en la ruta y tradúzcalo a una respuesta 409 con el formato uniforme, en lugar de dejar que llegue al manejador de 500.
MongoServerSelectionError: connect ECONNREFUSED 127.0.0.1:27017	El contenedor de la base no está corriendo, o dentro de Docker Compose se usó localhost en vez del nombre del servicio.	Verifique con docker ps; dentro del compose use mongodb://mongo:27017 y confirme que ambos servicios comparten la misma red.
Error: listen EADDRINUSE: address already in use :::3000	Quedó un proceso anterior escuchando en el mismo puerto, o el contenedor ya publica el 3000.	Cierre el proceso previo (lsf -i :3000 en Linux o macOS, netstat -ano en Windows) o cambie el valor de PORT.
Access to fetch has been blocked by CORS policy	Un cliente servido desde otro origen (un front en el puerto 5173) llama a la API sin que esta declare la política de origen cruzado.	Instale cors y agregue app.use(cors({ origin: "http://localhost:5173" })). No use origin: "*" en un servicio que maneje datos reales.
Cannot set headers after they are sent to the client	La ruta respondió dos veces: faltó el return antes de un res.status(...).json(...) de error.	Anteponga return a cada respuesta de error o use if/else. Una ruta responde exactamente una vez.
La respuesta del error llega en HTML y no en JSON	La excepción llegó al manejador por omisión de Express porque no se registró un manejador de errores propio.	Registre el middleware de cuatro parámetros (err, req, res, next) al final de app.js, después de todas las rutas.

Antes de pedir ayuda: Antes de pedir ayuda, ejecute la petición con curl -i y lea las tres primeras líneas: el código de estado, el Content-Type y el cuerpo. Si el código es 4xx el problema está en su petición; si es 5xx está en su código y el detalle aparece en la terminal del servidor. Ese solo hábito resuelve la mayoría de los casos.

Referencias

- RFC 9110 — HTTP Semantics (métodos y códigos de estado) — <https://www.rfc-editor.org/rfc/rfc9110>
- MDN — Códigos de estado de respuesta HTTP — <https://developer.mozilla.org/es/docs/Web/HTTP/Status>

- Documentación de Express — enrutamiento y manejo de errores — <https://expressjs.com/es/guide/routing.html>
- OpenAPI Specification 3.0.3 — <https://spec.openapis.org/oas/v3.0.3>
- Fowler, Martin. Richardson Maturity Model — <https://martinfowler.com/articles/richardsonMaturityModel.html>
- Richardson, Chris. Microservices Patterns, capítulo sobre APIs y comunicación entre servicios. Manning, 2018.
- Diapositivas del Tema 4 — Comunicación entre Microservicios (COM-600).